Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Guardians of the Chess Grandmaster

By
Tomás Pettit

April 25, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction of GOTCG **2**
1.1 Project Context 2
1.2 Project Objectives 3
1.3 Success Metrics 3

2 Methodology of Chess App **7**
2.1 Development Approach 7
2.2 Development Process Management 8
2.3 Technology Selection and Justification 8
2.4 Research Foundation 9
2.5 Problem-Solving Approach 10
2.6 Development Environment and Tools 10
2.7 Deployment and Limitations 10

3 Technology Review **12**
3.1 Frontend Technologies 13
3.2 Backend Technologies 15
3.3 Database Technologies 16
3.4 Development Standards 18
3.5 Third-Party Libraries 20
3.6 Technology Stack Integration 23

4 System Design of Chess App **27**
4.1 Overall System Architecture 27
4.2 Detailed Component Design 29
4.3 Real-Time Communication (Socket.IO) 32
4.4 Cloud Infrastructure & Deployment 33

5 System Evaluation of Testing Application **37**
5.1 Testing the Application 37
5.2 Limitations and Opportunities for Future Work 42

6 Conclusion of GOTCG **45**
6.1 Context and Objectives 45
6.2 Population of People Who Played GOTCG 45
6.3 Summary of Findings from System Evaluation 46
6.4 Opportunities for Future Investigation 47
6.5 Closing Remarks 48

List of Figures

2.1 PWA Chess App Logo Design 9
2.2 Four files on `Utils/` has been educated on web technologies 10
2.3 Two files of `.yml` for deploying both to initialised an app 11

3.1 Chess Board setup (Source: Chess.com) 14
3.2 MongoDB Document Example 17
3.3 Data Flow Diagram Example 24

4.1 My System Architecture Diagram 28

5.1 Frontend Automated Test Results 38
5.2 Backend Automated Testing Results 39

6.1 Chess.com World Map Population 46
6.2 Six AI difficulty levels, including a special level 48

List of Tables

3.1 MongoDB vs. Relational Database Comparison 18
3.2 RESTful API Structure 19

4.1 Technology Stack from `frontend` 29
4.2 Technology Stack from `backend` 30
4.3 API Endpoint Reference 31
4.4 Client to Server Socket Events 32
4.5 Server to Client Socket Events 33
4.6 Technology Selection Rationale 35

5.1 Test results for `tests/routes/` 39

Chapter 1

Introduction of GOTCG

1.1 Project Context

"Guardians of the Chess Grandmaster" is an **innovative** chess learning application designed to provide an engaging and accessible platform for chess **enthusiasts**, whether they are complete beginners or aspiring players looking to reach a more advanced level. The project focuses on teaching the **intricacies** of chess through a variety of interactive methods, such as puzzles, strategic missions, and real-time matches against other players or an AI opponent like practice for preparing for other players to face in.

Chess is often seen as a complex game that requires a deep understanding of strategies, tactics, and piece movements. Many **aspiring** players face challenges due to a lack of clear guidance, overwhelming information, or a decline in motivation. This application addresses these barriers by breaking down the game's concepts into manageable lessons that highlight both its strategic depth and **artistic** nature. It aims to transform the learning process into an enjoyable experience that encourages players to persist in their chess journey.

The scope of this project is suitable for an eighth-grade level and above, as it introduces chess concepts in a structured way while also being challenging enough for more advanced players. Overall, readers should find this project compelling due to its potential to make chess more accessible, enjoyable, and educational for a wide range of users. As chess continues gaining popularity worldwide, tools that simplify learning and promote engagement are essential for **fostering** a new generation of skilled players.

Before reading further, readers are encouraged to watch the short screencast demonstration of the application, available at GOTCG - Final Year Project - Screencast, to gain a practical sense of the platform in action prior to exploring the technical detail that follows.

1.2 Project Objectives

1. **User-Friendly Interface**: Design and develop a landing interface with **intuitive** login and sign-up options to ensure secure user authentication.

2. **Interactive Home Page**: Create a functional home page that provides easy access to all main features, including game modes, tutorials, and user profiles, with a focus on game history.

3. **Comprehensive User Profile**: Develop a user profile page where players can manage their personal information and view their game history.

4. **Customization Settings**: Implement a settings section that allows users to customize preferences such as themes, piece sets, cache management, and install an app while using the Progressive Web Application (PWA).

5. **In-App Tutorial Section**: Provide a tutorial section that effectively teaches users the basic and advanced rules of chess, including the functions of each piece and strategies for winning or drawing at the end of the game.

6. **Play Features**: Enable various game modes, including player vs. player (where one player creates a Room ID and the other can join using that Room ID) and player vs. AI. Differentiate the experiences through an easy naming process and difficulty options.

7. **Friends Feature**: Include functionality for users to manage their friends, send requests, challenge one another by inviting friends to play, and chat to check availability for games by sending or replying to messages.

8. **Secure Logout Functionality**: Implement a secure logout feature that safely ends user sessions and prevents access to protected content.

1.3 Success Metrics

Success or failure of the project will be measured using the following metrics:

- **User Authentication Success**: Successful login and registration rates, with effective password recovery tests showing a **low failure rate**.

- **Navigation and Feature Access**: High user engagement levels as measured by the number of users accessing the home page and successfully navigating to different features.

- **Profile Updates**: The number of successful profile updates and the correct display of changes made.
- **Preference Persistence**: Confirmation that user customization settings remain **intact** after app restarts.
- **Tutorial Accessibility and Utilization**: Monitoring tutorial access frequency and user completion rates.
- **Game Functionality**: Successful match **initiation rates** for all game modes with proper AI responses.
- **Friends Feature Efficiency**: Number of successful friend requests, chat messages and in-game challenges made.
- **Logout Security**: Tests confirming that users cannot access secured content post-logout.

By addressing these objectives and utilizing measurable success metrics, the project aims to create a comprehensive chess learning platform that is both functional and engaging for users.

Chapter/Section Overview

- **Methodology** - This section outlines the approach taken for the project, discussing the validity and rationale behind the chosen methods on *Chapter* 2.
- **Technology Review (Literature Review)** A detailed examination of the technologies used in the project on a Literature Review on *Chapter* 3:
  - **Frontend**: Utilizes React with Vite and TypeScript to create a responsive interface, leveraging efficient development processes and smooth updates.
  - **Backend**: Incorporates Node.js or Express.js to define server-side logic, manage authentication, and handle data storage.
  - **Database**: Employs MongoDB as a NoSQL database to manage user profiles and game data, using Mongoose for object data modeling and streamlined interactions.
  - **Operating System Compatibility**: The web-based application is compatible with any OS supporting modern browsers, emphasizing cross-platform development.

  - **Third-Party Libraries and Services**: Integrates various libraries to enhance functionality and simplify development.

- **System Design** - Provides a comprehensive overview of the system architecture, detailing the components and connections within the project in *Chapter* 4.

- **System Evaluation** - Assesses the project against the initial objectives set in the introduction, discussing successes and areas for improvement in *Chapter* 5.

- **Conclusion** - Summarizes the project context, key findings, and overall objectives on *Chapter* 6.

Resource URLs

- **GitHub Repository**: Project GitHub URL

- **Frontend Folder (React, Vite, and TypeScript)**: Frontend Link

- **Backend Folder (Node.js and Server.js)**: Backend Link

- **AI Model Development**: AI Model Development Link

- **Integration**: Integration Link

- **Documentation Folder**: Documentation Link

- **Work Plan**: Link to My Work Plan Projects

- **Screencast Video**: GOTCG - Final Year Project - Screencast

Main Elements at the URL

1. **Source Code** - The complete codebase for the project is organized into `frontend` (React + Vite) and `backend` (Node.js & Server.js) components, with a clear structure that facilitates easy navigation.

2. **Documentation** - A comprehensive `README` file is provided, which includes installation instructions, usage guidelines, and setup procedures for both development and deployment. Additionally, a brief screencast (2-5 minutes) showcasing your application in action is included to demonstrate its functionality.

3. **Deployments** - GitHub offers robust deployment capabilities through GitHub Actions, allowing for seamless integration and automation when deploying applications. Here are the steps to deploy your project using GitHub: the `frontend` can be deployed on Vercel, while the `backend` can be deployed on Railway.

4. **Issues Tracker** - There is a dedicated section for reporting bugs or requesting features, which encourages community engagement and feedback.

5. **Contributing Guidelines** - Instructions for how other developers can contribute to the project are included, covering coding standards and the submission process.

6. **License** - Information about the open-source license under which the project is distributed is provided, promoting transparency and encouraging community use.

Chapter 2

Methodology of Chess App

Introduction of Methodology

This chapter outlines the methodology employed in the development of *Guardians of the Chess Grandmaster* (GOTCG), an AI-enhanced chess learning platform. The approach combines software development practices with a **research-informed** methodology to create a comprehensive educational tool for chess players of varying skill levels.

2.1 Development Approach

The project adopted a **software development methodology** rather than a pure research methodology, prioritizing the creation of a functional, user-centered application that **leverages** artificial intelligence to enhance chess learning experiences.

Agile and Iterative Development

An **agile and incremental approach** was employed throughout the development lifecycle, enabling continuous refinement and adaptation based on testing outcomes and emerging requirements. This methodology **facilitated** rapid prototyping, continuous integration of user feedback, flexible response to technical challenges, and iterative improvement of core functionalities.

Planning and Requirements Analysis

Project requirements were determined through market research analyzing existing chess platforms (Chess.com, Lichess), user story development for different skill levels, feature mapping of core functionalities, and technical feasibility studies.

Requirements were documented and prioritized using a backlog system, with features categorized by their importance to the minimum viable product (MVP) and subsequent iterations.

2.2 Development Process Management

Version Control and Collaboration

GitHub was utilized as the primary development tool, providing version control through Git, issue tracking for bug reporting and feature requests, project boards for task management, code review capabilities through pull requests, and documentation hosting. The repository was organized into distinct `/frontend` and `/backend` directories.

Development Workflow

Development followed a structured workflow: feature planning translated **user stories** into technical tasks; implementation occurred in feature branches with regular commits; testing validated functionality locally; code review ensured quality and **adherence** to standards; integration merged approved changes; and deployment released updates to production.

2.3 Technology Selection and Justification

Frontend Technologies

The `frontend` architecture employed **React 18.3.1** for its component-based architecture and efficient state management; **TypeScript 5.6.2** to enhance code quality through static typing; **Vite** for fast Hot Module Replacement during development; **Material-UI** for consistent interface design; and **Framer Motion** for smooth animations.

Backend Technologies

The `backend` infrastructure utilized **Node.js (v22)** for its non-blocking I/O model essential for concurrent connections; **Express.js** as the web application framework; **Socket.IO** for bidirectional real-time communication; **Firebase** for authentication and user profiles; and **MongoDB** for game data storage due to its flexible schema design.

Figure 2.1: PWA Chess App Logo Design

Progressive Web App (PWA) Implementation

The platform was developed as a PWA to provide cross-device compatibility, offline functionality through service workers, installability without app store distribution, and fast loading times. There are two things for deploying both `frontend` and `backend` → Vercel and Railway. An example on Figure 2.1

Validation and Testing Strategy

A comprehensive testing strategy was implemented using **unit testing** to verify individual components; **integration testing** for API endpoints and database interactions; **end-to-end testing** for complete user workflows; and **real-time testing** for WebSocket reliability. Validation included functional testing of all features, performance monitoring, usability evaluation, cross-browser compatibility testing, and responsive design verification across devices.

2.4 Research Foundation

Prior to implementation, research was conducted on chess rating systems (Elo algorithm), chess learning methodologies, existing platform analysis, AI integration in education, and real-time web technologies. This research informed design decisions and feature priorities throughout development. For example on Figure 2.2

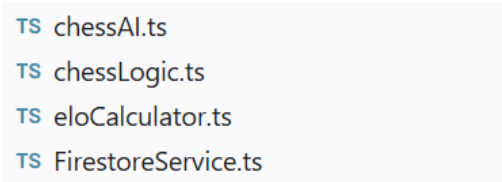

Figure 2.2: Four files on `Utils/` has been educated on web technologies

2.5 Problem-Solving Approach

An **empirical approach** was employed when addressing technical challenges. Problems were logged and categorized by severity; debugging tools and systematic testing identified root causes; multiple solutions were prototyped and evaluated; selected solutions were implemented and validated; and lessons learned were documented. Key challenges included real-time synchronization across clients, Firebase authentication integration, **hybrid** database architecture design, PWA configuration, and development environment consistency.

2.6 Development Environment and Tools

The development environment consisted of Node.js v22, npm for package management, Firebase CLI for integration, Git for version control, and Visual Studio Code with relevant extensions. Environment variables were managed through `.env` files for Firebase configuration, MongoDB connection strings, API endpoints, and service credentials, ensuring sensitive data remained secure while maintaining consistency across environments.

2.7 Deployment and Limitations

The platform was deployed as a Progressive Web App with `frontend` static file serving, `backend` API and WebSocket hosting, Firebase authentication services, and MongoDB Atlas for data storage. Limitations included time **constraints** affecting feature scope, limited access to extensive user testing groups, reliance on third-party services, and single-developer capacity constraints. Here's an example from Figure 2.3

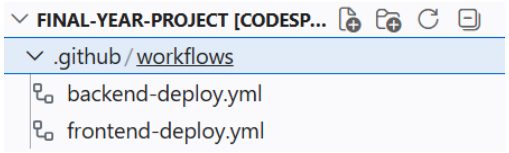

Figure 2.3: Two files of `.yml` for deploying both to initialised an app

Summary

This methodology chapter has outlined the comprehensive approach taken in developing *Guardians of the Chess Grandmaster*. The agile, iterative development process, combined with careful technology selection and systematic testing, enabled the creation of a functional AI-enhanced chess learning platform. The **empirical** problem-solving approach and research-informed decision-making ensured that technical challenges were addressed effectively while maintaining focus on user needs and educational objectives.

Chapter 3

Technology Review

Introduction of Technology Review

This chapter provides a comprehensive review of the technologies, frameworks, and services employed in the development of "*Guardians of the Chess Grandmaster*," a web-based chess training platform designed to challenge players and develop grandmaster-level thinking skills. The technology review demonstrates the research and evaluation conducted to select appropriate tools and approaches for building a modern, scalable, and performant web application.

As established in the introduction, the primary objectives of this project are to:

- Provide an interactive chess training platform with puzzle-based learning
- Deliver real-time feedback on player moves and positions
- Track user progress and performance analytics
- Ensure responsive performance across desktop and mobile devices
- Maintain scalability for growing user base and puzzle database

To achieve these objectives, each technology selection was evaluated based on specific criteria relevant to chess training applications: real-time interactivity, data structure flexibility for chess positions, performance optimization for board rendering, and maintainability for ongoing development. This review examines technologies at a conceptual level, with justification based on project requirements, industry standards, and academic literature.

The review is structured to cover all major components of the technology stack:

- Frontend technologies (React, TypeScript, Vite)

- Backend infrastructure (Node.js, Express.js)
- Database systems (MongoDB)
- Development standards (REST, JSON)
- Third-party libraries (Chess.js, React-Chessboard)

Each section examines the conceptual foundations of the technology, its key features, and the rationale for its selection in the context of building an effective chess training platform.

3.1 Frontend Technologies

The `frontend` is the primary interface between users and the system. For a chess training platform, `frontend` technology choices directly affect board responsiveness, piece movement smoothness, and the overall puzzle-solving experience.

React Framework

React is a declarative, component-based JavaScript library developed by Meta for building user interfaces. Unlike full-stack frameworks such as Angular or Vue, React focuses on the view layer, giving developers architectural flexibility for other concerns [1] (React Documentation: react.dev).

React's architecture rests on three core concepts. First, **component-based architecture** allows applications to be built from modular, reusable components that encapsulate structure, styling, and behaviour. For a chess application, the board, individual pieces, move history panel, and puzzle instructions can each be isolated components. A `ChessBoard` component might contain 64 `Square` components, each potentially housing a `Piece` component and the coordinates (a-h, 1-8), as illustrated in Figure 3.1.

Second, React's **virtual DOM and reconciliation** mechanism avoids costly direct DOM manipulation. When state changes, React compares a new virtual DOM tree against the previous version and applies only the minimal set of real DOM updates [2]. For a chess application where piece movements and board state changes occur frequently, this prevents unnecessary full re-renders and maintains smooth interactions.

Third, React enforces **unidirectional data flow**: data flows downward via props and state changes propagate upward through callbacks [3]. This predictability simplifies debugging and testing. When a user moves a piece, the event flows upward to the parent component managing game state, which validates the move

Figure 3.1: Chess Board setup (Source: Chess.com)

via Chess.js, updates state, and propagates the new position down to child components for rendering.

React was selected for this project due to its performance for interactive applications [2], strong component reusability across training modes (tactical puzzles, endgame training, opening practice), its rich ecosystem (React Router, Context API, UI libraries), and its position as the most widely used web framework per the Stack Overflow Developer Survey 2023 (Stack Overflow, 2023). Critically, the chess-specific library React-Chessboard is built for React, enabling seamless integration.

TypeScript

TypeScript is a statically-typed superset of JavaScript developed by Microsoft that adds optional type annotations and compiles to standard JavaScript [4] (TypeScript Documentation: typescriptlang.org).

TypeScript's **static type system** allows explicit declaration of variable types, function parameters, and return values, with the compiler catching mismatches before runtime. **Type inference** reduces verbose annotations while maintaining safety (TypeScript Handbook), and **interfaces** define contracts for object shapes that serve simultaneously as type-checking mechanisms and inline documentation.

TypeScript was selected for several reasons relevant to this project. Research by Gao et al. [5] found TypeScript detects approximately 15% of bugs that would otherwise surface at runtime—a significant benefit for complex chess logic such as

move validation, position evaluation, and rating calculations. TypeScript also provides superior IDE support with IntelliSense and autocomplete for chess.js library methods. For a project evolving over multiple months, its compile-time safety makes refactoring safer, and its type annotations enforce API contract consistency between `frontend` and `backend`.

Vite Build Tool

Vite is a modern `frontend` build tool created by Evan You that prioritises developer experience through instant server startup and rapid hot module replacement (Vite Documentation).

Traditional bundlers such as Webpack must bundle the entire application before the development server starts, becoming slow as codebases grow. Vite addresses this by serving source code as native ES modules during development—leveraging native browser ES module support (ECMAScript Modules)—and only transforming files on-demand as the browser requests them. When source files change, Vite's HMR invalidates only the changed module, preserving application state and delivering near-instantaneous browser updates (Vite Features). For production, Vite uses Rollup (Rollup) to apply tree-shaking, code splitting, and minification.

Vite was selected for its development velocity—iterating on chess puzzles and training modes benefits greatly from rapid feedback loops. Benchmarks show Vite achieving 10–100× faster cold server starts and 2–10× faster HMR compared to Webpack [6]. It also provides first-class support for React and TypeScript through official plugins, requiring no complex configuration.

3.2 Backend Technologies

The `backend` serves as the application's server-side logic layer, handling data processing, business rules, authentication, and database communication. For a chess training platform, `backend` requirements include puzzle delivery, move validation, user progress tracking, and rating calculations.

Node.js Runtime Environment

Node.js is a JavaScript runtime built on Chrome's V8 engine, enabling JavaScript execution outside the browser. Created by Ryan Dahl in 2009, it brought JavaScript to server-side development, enabling full-stack JavaScript applications (About Node.js).

Node.js employs an **event-driven, non-blocking architecture** centred on an event loop [7]. Rather than spawning a new thread per connection, the single-

threaded event loop accepts requests, delegates I/O operations to the system, continues processing other requests without blocking, and executes callbacks on completion. This allows Node.js to handle thousands of concurrent connections with minimal memory overhead, well-suited to the I/O-intensive nature of a chess training server [8]. The V8 engine compiles JavaScript to native machine code (V8 Docs), and the npm ecosystem provides access to over 2 million reusable packages (npm Registry).

Node.js was selected principally for **JavaScript uniformity**: using a single language across `frontend` and `backend` eliminates context switching and enables sharing of move validation logic, FEN parsing utilities, and notation converters between client and server [9]. It also handles JSON natively, simplifying data exchange between MongoDB and the React `frontend` [10], and provides strong real-time capabilities for potential future features such as live multiplayer analysis and collaborative puzzle solving [11].

Express.js Framework

Express.js is a minimal, unopinionated web framework for Node.js, first released in 2010 and now the de facto standard for Node.js web development (Express Documentation).

Express applications are composed of **middleware functions** with access to the request and response objects and the next function in the request-response cycle [12]. Middleware can execute code, modify request/response objects, end the cycle by sending a response, or pass control to the next middleware—promoting modularity and code reuse. Its flexible **routing** supports HTTP method handlers, route parameters (e.g. `/puzzles/:id`), query parameters, and modular route grouping. Unlike opinionated frameworks such as Django or Rails, Express's minimalism allows developers to choose appropriate tools for specific requirements without working around framework constraints [13].

Express was selected for its lightweight foundation, its natural fit with RESTful API design, and its extensive middleware ecosystem (e.g. `cors`, `helmet`, `morgan`, `express-rate-limit`). It is production-proven at scale, used by organisations including Uber and IBM (Node.js Case Studies).

3.3 Database Technologies

MongoDB NoSQL Database

MongoDB is a **document-oriented** NoSQL database that stores data in flexible, JSON-like documents called BSON (Binary JSON) MongoDB Manual. Released in

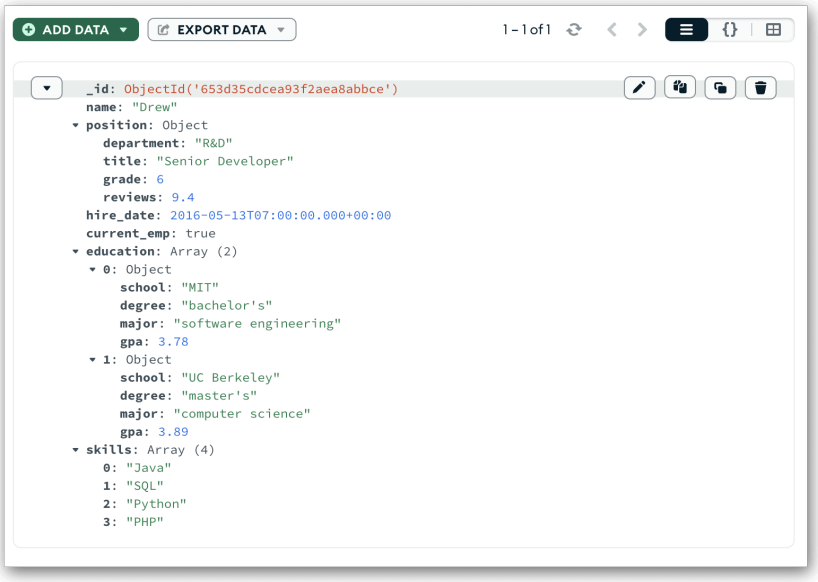

Figure 3.2: MongoDB Document Example

2009, MongoDB represents a paradigm shift from traditional relational databases toward schema-flexible document storage.

Conceptual Foundation

Document Model → Unlike relational databases that store data in tables with fixed schemas, MongoDB stores data in documents—self-contained data structures that can include nested objects and arrays [14]. A chess puzzle document might look like 3.2:

This document encapsulates all puzzle-related information in a single, hierarchical structure.

Schema Flexibility → MongoDB's schema-less design allows documents in the same collection to have different structures [15]. This flexibility benefits chess applications where:

- Different puzzle types may have **different attribute**s (endgame puzzles might include piece count, opening puzzles might reference ECO codes)
- New features can add fields without database **migrations**
- Experimental features can store **additional data** without affecting existing documents

Indexing and Query Optimization → MongoDB supports various index types to optimize query performance MongoDB Indexes:

- Single field indexes (e.g., on `difficulty`)
- Compound indexes (e.g., on `difficulty` and `rating`)
- Multi-key indexes for arrays (e.g., indexing `themes` array)
- Text indexes for full-text search

For a chess platform, indexes on `difficulty`, `themes`, and `rating` enable efficient puzzle filtering and recommendation queries.

Aggregation Framework → MongoDB's aggregation pipeline processes documents through multiple stages to compute analytics MongoDB Aggregation.

Comparison with Relational Alternatives

Relational databases (PostgreSQL, MySQL) were considered but ultimately not selected:

Aspect	**Relational DB**	**MongoDB**
Schema	Fixed, requires migrations	Flexible, evolves naturally
Chess positions	Difficult to normalize, requires complex joins	Natural document structure
Querying nested data	Complex joins across tables	Simple dot notation
Performance for reads	Join overhead for related data	Single document retrieval
Development velocity	Slower due to migrations	Faster with schema flexibility
JSON handling	Requires ORM mapping	Native JSON/BSON

Table 3.1: MongoDB vs. Relational Database Comparison

For this project's requirements—flexible schema, hierarchical chess data, read-heavy workload, and rapid iteration—MongoDB's document model provides superior fit [16].

3.4 Development Standards

REST Architectural Style

REST (Representational State Transfer) is an architectural style for networked applications introduced by Roy Fielding in his 2000 doctoral dissertation [17]. RESTful APIs use standard HTTP methods and conventions to perform operations on resources.

Puzzle endpoints	
HTTP Method	Endpoint
GET	`/api/puzzles?difficulty=hard&theme=fork`
POST	`/api/puzzles`
GET	`/api/puzzles/:id`
PUT	`/api/puzzles:id`
DELETE	`/api/puzzles/:id`

Table 3.2: RESTful API Structure

Core Principles

Resource-Based → RESTful APIs organize around resources (nouns) rather than actions (verbs) [18]. Resources are identified by URIs:

- `/api/puzzles` represents the puzzle collection
- `/api/puzzles/123` represents a specific puzzle
- `/api/users/456` represents a specific user

Standard HTTP Methods → Operations on resources use standard HTTP methods with defined semantics [19].

Statelessness → Each request contains all information needed for processing; no session state is stored on the server [17]. Benefits include:

- Simplified server implementation
- Easy horizontal scaling (any server can handle any request)
- Improved reliability (no session loss on server restart)

For this platform, authentication uses JWT tokens sent with each request, maintaining statelessness.

Implementation in Chess Platform

The API follows RESTful conventions 3.2:

This design provides:

- Intuitive endpoint structure
- Clear HTTP method semantics
- Easy integration with `frontend` frameworks
- Self-documenting API structure

JSON Data Format

JSON (JavaScript Object Notation) is a lightweight, text-based data interchange format [20]. It has become the standard for web APIs, superseding XML in most modern applications.

Advantages

1. **Native JavaScript Support**: JSON parses to JavaScript objects without libraries, reducing overhead [10].

2. **Human Readable**: JSON's syntax is intuitive, facilitating debugging and manual API testing.

3. **Compact Format**: Research by Nurseitov et al. [10] found JSON payloads are approximately 30% smaller than equivalent XML, reducing bandwidth and improving load times.

4. **Language Independent**: While based on JavaScript notation, JSON has parsers in all major programming languages [21].

Use in Project

JSON is used throughout the stack:

- API request and response bodies
- MongoDB storage (as BSON, a binary superset of JSON)
- Configuration files (`package.json`, `tsconfig.json`)
- State management in React

3.5 Third-Party Libraries

Chess.js

Chess.js is a JavaScript library for chess move generation, validation, and game state management chess.js. It provides comprehensive chess rule implementation complying with FIDE standards.

Key Capabilities

- **Move Generation**: Generates all legal moves for any position, accounting for checks, pins, castling rights, and en passant
- **Move Validation**: Ensures only legal moves are played, preventing invalid game states
- **FEN Support**: Loads and exports positions in Forsyth-Edwards Notation, the standard for position representation
- **PGN Support**: Reads and writes games in Portable Game Notation for interoperability with other chess software
- **Game State Detection**: Identifies check, checkmate, stalemate, threefold repetition, and fifty-move rule
- **Move History**: Maintains complete move history with undo capabilities

Justification

Implementing chess rules from scratch would be error-prone and time-consuming. Chess has numerous special cases and edge conditions:

- Castling (legality depends on king/rook movement history, intervening squares, check status)
- En passant (only legal immediately after opponent's pawn double-move)
- Pawn promotion
- Check and checkmate detection
- Stalemate and draw conditions
- Pin mechanics (pieces cannot move if doing so exposes king to check)

Chess.js provides a thoroughly tested implementation, allowing the project to focus on training features rather than rule implementation. The library has:

- Over 1.3 million weekly npm downloads chess.js package
- Comprehensive test suite
- Active maintenance since 2011
- Wide adoption in chess applications

React-Chessboard

React-Chessboard is a customizable chessboard component for React applications. It provides visual board rendering with interactive piece movement react-chessboard

Features

- **Drag-and-Drop**: Mouse and touch-based piece movement
- **Customization**: Configurable piece sets, board colors, and square styles
- **Position Control**: Accepts FEN strings for position display
- **Event Callbacks**: Handlers for piece drops, square clicks, and other interactions
- **Responsive**: Scales appropriately for different screen sizes
- **Accessibility**: Keyboard navigation support

Justification

Building a chess board UI from scratch requires significant implementation effort:

- SVG rendering for pieces (12 piece types with multiple design options)
- Drag-and-drop interaction with collision detection
- Touch event handling for mobile devices
- Responsive sizing maintaining 1:1 aspect ratio
- Animation for piece movements
- Square highlighting and annotations

React-Chessboard provides a production-ready, accessible solution, allowing focus on puzzle presentation logic, feedback mechanisms, and progress tracking rather than low-level UI implementation.

Authentication Libraries

JSON Web Tokens (JWT)

JWT (RFC 7519) is a compact, URL-safe means of representing claims between two parties [22]. JWTs enable stateless authentication where the server doesn't maintain session storage.

Structure:

- **Header**: Algorithm and token type
- **Payload**: Claims (user ID, roles, expiration)
- **Signature**: Cryptographic signature for verification

Benefits for Chess Platform:

- Stateless authentication (no server session storage)
- Scalable across multiple servers
- Self-contained (includes user identity and permissions)
- Standard format with libraries in all languages

For user account security, bcrypt provides industry-standard password protection, ensuring that even if the database is compromised, passwords remain protected against offline attacks.

3.6 Technology Stack Integration

The selected technologies integrate to form a cohesive, modern web application architecture. Figure 3.3 illustrates the data flow diagram.

Data Flow Example

A typical user interaction demonstrates technology integration:

1. User clicks to load a puzzle (React component event)
2. Frontend sends GET request to `/api/puzzles/:id` (REST, JSON)
3. Express router receives request, validates JWT token
4. Backend queries MongoDB for puzzle document

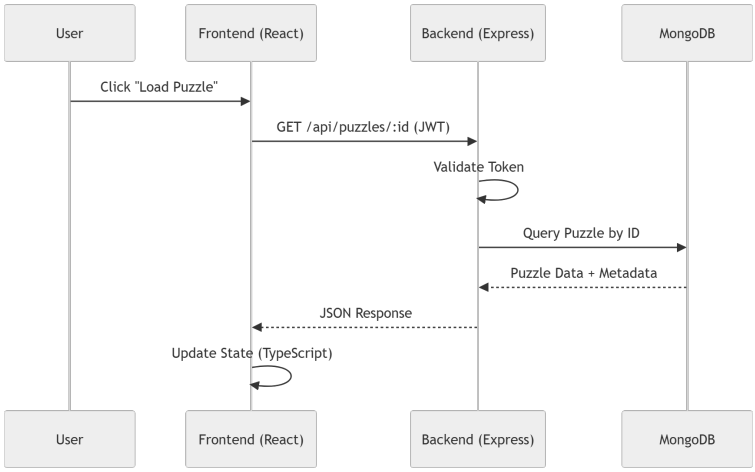

Figure 3.3: Data Flow Diagram Example

5. MongoDB returns puzzle with embedded solution and metadata
6. Backend sends JSON response to `frontend`
7. React state updates with puzzle data (TypeScript interfaces ensure type safety)
8. React-Chessboard renders position from FEN string
9. User makes move via drag-and-drop
10. Chess.js validates move legality
11. If correct, `frontend` shows success feedback
12. Backend receives solution attempt POST request
13. MongoDB updates user statistics and puzzle attempt count

This flow demonstrates seamless integration across all technology layers.

Technology Stack Benefits

The integrated stack provides:

1. **Performance**
    - React virtual DOM optimizes rendering

- Node.js event loop handles concurrent users efficiently
- MongoDB indexes optimize puzzle queries
- Vite HMR accelerates development iteration

2. **Scalability**
   - Stateless REST API enables horizontal scaling
   - MongoDB supports sharding for data distribution
   - JWT authentication requires no server-side session storage
3. **Maintainability**
   - TypeScript catches errors at compile-time
   - Component architecture promotes code reuse
   - Clear separation of concerns (frontend/backend/database)
   - Shared JavaScript enables code reuse across stack
4. **Developer Experience**
   - Single language (JavaScript/TypeScript) across stack
   - Rich ecosystem of libraries and tools
   - Excellent IDE support with autocomplete and type checking
   - Fast development feedback loops

Conclusion

This technology review has examined the complete technology stack for "Guardians of the Chess Grandmaster," demonstrating thorough research and justified selection based on project requirements, performance characteristics, and academic literature.

The chosen stack—React with TypeScript for `frontend`, Node.js with Express for `backend`, and MongoDB for database—provides a robust, modern foundation for building a chess training platform that meets the objectives outlined in the introduction:

- **Interactive Training**: React and React-Chessboard enable smooth, responsive board interactions
- **Real-Time Feedback**: Chess.js provides instant move validation

- **Progress Tracking**: MongoDB's flexible schema accommodates evolving user statistics
- **Cross-Device Support**: Responsive React components and mobile-friendly design
- **Scalability**: Stateless architecture and efficient I/O handling support growth

Each technology selection was justified through:

- Alignment with project-specific requirements
- Performance characteristics relevant to chess applications
- Industry adoption and community support
- Academic research demonstrating effectiveness
- Standards compliance ensuring interoperability

The technology review establishes the technical foundation upon which subsequent chapters build system design, implementation details, and evaluation. The selection of mature, well-documented, industry-standard technologies minimizes technical risk while providing the flexibility needed for iterative development and future feature enhancement.

Chapter 4

System Design of Chess App

Introduction of System Design

This section presents the detailed architectural design of **Guardians of the Chess Grandmaster (GOTCG)**, a web application platform designed to challenge and train players to think like a grandmaster. The design follows a client-server architecture with a React-based `frontend` and Node.js/Express `backend`, integrated with both Firebase and MongoDB for authentication, real-time data, and game analytics.

The architecture has been informed by the technology review conducted in the previous section, emphasizing scalability, real-time communication, and a responsive user experience across devices.

4.1 Overall System Architecture

System Architecture Diagram

The system follows a modern full-stack web architecture, with a clear separation of concerns across the `frontend`, `backend`, and data layers. The `frontend` is responsible for user interaction and presentation, providing a responsive and dynamic user interface. The `backend` manages core application logic, authentication, and real-time communication between clients. Data persistence is handled using both Firebase and MongoDB, with each technology serving different requirements depending on the use case. This hybrid approach enables efficient user management alongside scalable storage for game state and analytics data. Here's an example from Figure 4.1

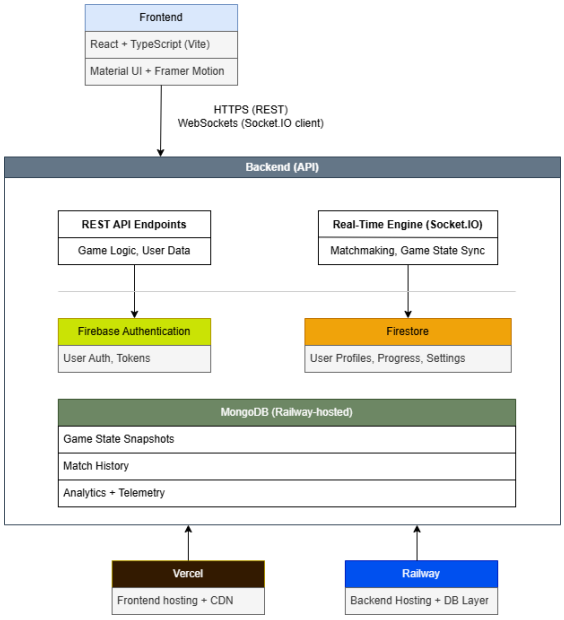

Figure 4.1: My System Architecture Diagram

High-Level Architecture

The GOTCG platform employs a **three-tier architecture**:

1. **Presentation Layer**: React SPA (Single Page Application) with TypeScript
2. **Application Layer**: Node.js/Express API server with Socket.IO for real-time features
3. **Data Layer**: Dual database strategy - Firebase (authentication & Firestore) + MongoDB (game data & analytics)

Component Coupling & Communication Standards

Standards-Based Integration

- **REST API**: HTTP/HTTPS communication between `frontend` and `backend`
- **WebSocket Protocol**: Socket.IO for bi-directional real-time communication
- **JSON Data Format**: All API payloads use JSON for data interchange
- **Firebase SDK**: Official Firebase client library for authentication

Technology	Purpose
React	Language
TypeScript	Application framework
Vite	Authentication & authorisation
Firebase	Database access (Hibernate)
HTML	Relational database
CSS	Stateless auth tokens

Table 4.1: Technology Stack from `frontend`

- **Mongoose ODM**: Object-Document Mapping for MongoDB schemas

Component Coupling

- **Frontend** ↔ **Backend**: Loosely coupled via REST API contracts and WebSocket events
- **Backend** ↔ **Databases**: Abstracted through configuration modules (`/backend/src/config/`)
- **State Management**: React Context API for global state (Auth, Theme, Board Theme)

4.2 Detailed Component Design

Frontend Architecture (Presentation Layer)

Technology Stack

The `frontend` is built using modern web technologies as defined in Table 4.1

Frontend Structure

The `frontend` follows a layered architecture pattern following this link and looks for `Project Structure` GOTCG Frontend Project Structure

Key Frontend Components

The main application component (`App.tsx`) implements route-based navigation with authentication guards Route Configuration

Technology	Purpose
Node.js	Runtime environment
TypeScript	Language
Express.js	Web framework
MongoDB (via Mongoose)	Database
Firebase Admin SDK	Authentication & notifications
Jest	Testing framework
Docker	Containerization (optional, for deployment)

Table 4.2: Technology Stack from `backend`

Firebase Integration

Firebase is configured for authentication and Firestore database access Firebase Configuration

State Management Pattern

The application uses React Context API for global state management with a hierarchical provider structure Context Provider Hierarchy

This pattern ensures:

- **AuthProvider**: Manages user authentication state
- **ThemeProvider**: Handles light/dark theme preferences
- **BoardThemeProvider**: Controls chess board appearance

Backend Architecture (Application Layer)

Technology Stack

The `backend` leverages Node.js with the following table on Table 4.2

Backend Structure

The `backend` follows a layered architecture pattern following this link and look for `Project Structure` GOTCG Backend Project Structure

Server Configuration

The server entry point (`index.ts`) configures Express, CORS, and Socket.IO Server Configuration

API Endpoint Structure

The RESTful API exposes the following endpoint hierarchy 4.3:

API Endpoint Reference		
Endpoint	**Method**	**Description**
`/user/:id`	GET	Get user profile
`/user`	POST	Create user
`/user/:id`	PUT	Update user
`/game/:id`	GET	Get game details
`/game`	POST	Create new game
`/game/:id`	PUT	Update game state
`/friend/:userId`	GET	Get friends list
`/friend/add`	POST	Send friend request
`/friend/:friendId`	DELETE	Remove friend
`/game-invite/:id`	GET	Get game invites
`/game-invite`	POST	Send game invite
`/request/:userId`	GET	Get pending requests
`/request/respond`	POST	Accept/reject request
`/message/:chatId`	GET	Get chat messages
`/message`	POST	Send message

Table 4.3: API Endpoint Reference

Data Layer Architecture

Dual Database Strategy

The system employs two complementary database systems:

1. **Firebase (Authentication & Real-time Data)**
   - **Firebase Authentication**: Email/password + Google Sign-In
   - **Firestore**: User profiles, preferences, achievements
   - **Use Case**: User authentication, session management
2. **MongoDB (Game Data & Analytics)**
   - **Connection**: MongoDB Atlas cloud-hosted
   - **Use Case**: Game states, move history, analytics, statistics

4.3 Real-Time Communication (Socket.IO)

WebSocket Event Handlers

Socket.IO manages real-time bidirectional communication for:

- **Game Moves**: Broadcasting chess moves to both players
- **Game Invitations**: Real-time friend game invites
- **Chat Messages**: In-game messaging
- **Room Management**: Creating/joining game rooms

Room-Based Architecture

The `backend` maintains a room-based structure for game sessions, allowing one player to create a Room ID (six random digits), and when another player joins the same Room ID, it is the same as Player 1's.

Socket Events

- **Client → Server Events:** This enables the client to join a game room, send chess moves, chat with friends, and leave the game room if they have checkmated an opponent, resigned, timed out, or abandoned the game. Refer to Table 4.4 for examples.
- **Server → Client Events:** This allows the server to notify the client about game creation, broadcast moves to the opponent (client), share chat messages with friends, and notify the client of the game's conclusion. Refer to Table 4.5 for examples.

Client → Server Events	
Event	**Description**
joinRoom	Join a game room
makeMove	Send chess move
sendMessage	Send chat message
leaveRoom	Leave game room

Table 4.4: Client to Server Socket Events

Server → Client Events	
Event	**Description**
gameCreated	Notify game creation
moveMade	Broadcast move to opponent
messageReceived	Broadcast chat message
gameEnded	Notify game conclusion

Table 4.5: Server to Client Socket Events

4.4 Cloud Infrastructure & Deployment

Development Environment

- **Frontend**: Vite dev server (Ports 5173, 5174, 5175 for multi-instance testing)
- **Backend**: Node.js with nodemon (Ports 8000, 8001, 8002)

Database Hosting

- **Firebase**: Google Cloud Platform (GCP) - Authentication, Firestore
- **MongoDB**: MongoDB Atlas (Cloud-hosted DBaaS)

Progressive Web App (PWA)

The application leverages `vite-plugin-pwa` for:

- Offline support through service worker caching
- Installation to device home screen
- Native app-like experience across platforms
- Notifications allow apps to send alerts to users even when the app is closed. For example, send or reply a message and send a request to be a friend

Deployment Strategy (SaaS/PaaS)

To deploy a SaaS application using Vercel for the `frontend` and Railway for the `backend` (Warning: the `backend` should not be deployed to Vercel because Socket.IO/WebSocket support is required), consider the following steps:

1. **Vercel:**

- Host the `frontend` on Vercel, which is optimized for Next.js and other frameworks. It offers features like serverless functions, global CDN, and easy deployment via Git integration.
- Use Vercel's deployment hooks for automated deployments and preview URLs for testing.

2. **Railway:**

   - Deploy the `backend` on Railway, which is a container-focused PaaS that supports various frameworks and offers predictable billing.
   - Utilize Railway's features like one-click deploys and built-in database support for full-stack applications.

Security Design

Authentication & Authorization

- **Firebase Authentication**: JWT-based token authentication
- **Google OAuth 2.0**: Third-party authentication provider
- **CORS Configuration**: Whitelist of allowed origins

Data Security

- Environment variables for sensitive credentials (`.env` files)
- HTTPS/TLS for all production traffic
- Firebase Admin SDK for server-side authentication verification
- Password hashing handled by Firebase Authentication

Design Rationale

The architecture choices were informed by the technology review conducted in the previous chapter. Key decisions include 4.6:

Technology	Rationale
React + TypeScript	Type safety, component reusability, modern development experience
Vite	Fast HMR (Hot Module Replacement), optimized production builds
Material-UI	Consistent, accessible UI components following Material Design principles
Socket.IO	Cross-browser WebSocket support with automatic fallback mechanisms
Firebase	Scalable authentication, real-time database capabilities, managed infrastructure
MongoDB	Flexible schema for complex game data, excellent query performance for analytics
Node.js/Express	JavaScript full-stack development, extensive ecosystem, event-driven architecture
PWA	Offline capabilities, native app-like experience, cross-platform compatibility and notification

Table 4.6: Technology Selection Rationale

Scalability Considerations

The chosen architecture supports horizontal scaling through:

- **Stateless Backend**: Express servers can be replicated without session affinity
- **Socket.IO Redis Adapter**: Enables WebSocket connection sharing across multiple server instances
- **Cloud Databases**: Firebase and MongoDB Atlas automatically scale with demand
- **CDN Distribution**: Static `frontend` assets served globally through CDN

Performance Optimization

Performance is optimized through:

- **Code Splitting**: React lazy loading for route-based components
- **Service Worker Caching**: PWA caches static assets for faster subsequent loads

- **Database Indexing**: MongoDB indexes on frequently queried fields (`userId`, `gameId`)
- **WebSocket Compression**: Socket.IO compression reduces message payload sizes

Conclusion

This system design presents a comprehensive, scalable architecture for the *GOTCG* platform. The three-tier architecture with dual database strategy balances real-time requirements (Firebase) with complex data analytics needs (MongoDB). The use of modern web standards (REST, WebSocket, OAuth 2.0) ensures compatibility and maintainability, while the PWA approach provides a native-like experience across devices.

The design has been validated through implementation, as evidenced by the running codebase in the `tomaspettit2506/final-year-project` repository. Future enhancements could include AI-powered move suggestions, tournament management systems, and advanced analytics dashboards for player improvement tracking.

Chapter 5

System Evaluation of Testing Application

This chapter evaluates the *Guardians of the Chess Grandmaster* web application against the objectives established in *Chapter* 1. It presents the results of automated testing across the `frontend` and `backend`, discusses the deployment of the system to cloud platforms, assesses how well the implemented system meets its functional objectives, and reflects critically on the limitations of the approach and opportunities for future improvement.

5.1 Testing the Application

A comprehensive suite of automated tests was written to verify the correctness and robustness of the system. Testing was divided into two primary concerns: the `frontend` React application and the `backend` Node.js/Express.js API. The pages and components tested cover the full surface area of the application, spanning the following major views:

- **Landing** → unauthenticated entry point and sign-in flow
- **Home** → authenticated dashboard and navigation hub
- **Play** → chess game interface including board, timer, and move history
- **Tutorial** → interactive chess learning content
- **Friends** → social features including friend requests, challenges, and chat
- **Profile** → user statistics and game history
- **Settings** → account and preference management

Frontend Testing

The `frontend` was built using **React** with **TypeScript**, and all component and page tests were written using the **Jest** testing framework together with **React Testing Library**. Tests were executed by running the following command from the project root `cd frontend && npm run test`.

As shown in Figure 5.1, the test runner reported **39 test files** and **132 individual tests**, all of which passed. Tests were organised across two top-level directories: `Components/` and `Pages/`.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS 4

@tomaspettit2506 →/workspaces/final-year-project/frontend (main) $ npm run test
 Test Files  39 passed (39)
      Tests  132 passed (132)
   Start at  12:42:41
   Duration  82.79s (transform 2.03s, setup 14.41s, import 21.18s, tests 12.66s, environment 28.15s)

PASS Waiting for file changes...
       press h to show help, press q to quit
```

Figure 5.1: Frontend Automated Test Results

Component Tests

The `Components/` directory was subdivided into three feature-specific folders, each tested independently.

Page Tests

In addition to component-level tests, each of the seven application pages was tested independently to verify that routing, rendering, and top-level behaviour operated correctly.

Discussion of Frontend Results

The 100% pass rate across all 132 `frontend` tests demonstrates that the React components and pages render correctly, respond to user interactions as expected, and handle edge cases including empty states and loading conditions. The breadth of coverage — spanning the chess board, game controls, timer, move history, promotion dialogs, tutorial content, friend management, and PWA installation prompt — provides strong confidence in the structural correctness of the UI layer. That said, the tests are primarily unit and shallow-render tests; they confirm that components behave as specified in isolation but do not verify real browser interactions end-to-end. Full end-to-end testing using a tool such as **Cypress** or **Playwright**

would provide additional assurance that complete user journeys function correctly in a deployed environment.

Backend Testing

The `backend` was implemented using **Node.js** with the **Express.js** framework, with **Jest** used as the test runner for `backend` tests as well. Tests were executed using the following terminal command `cd backend && npm run test`

As shown in Figure 5.2, the test runner reported **9 test suites passed (9 total)** and **116 tests passed (116 total)**, with zero failures. This confirms that the `backend` API routes, middleware, and supporting logic behave correctly across all tested scenarios.

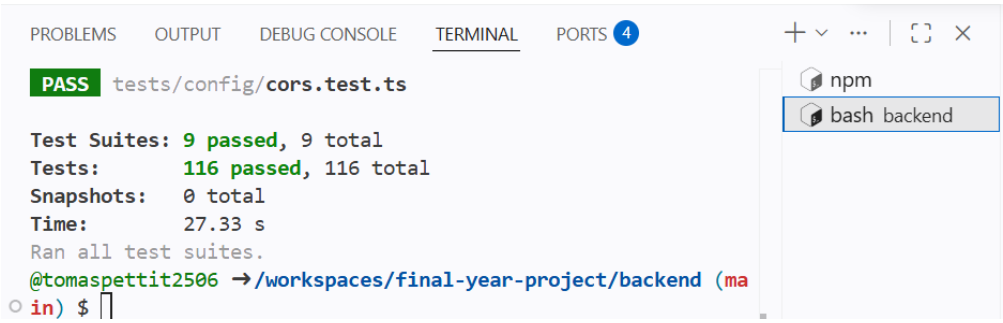

Figure 5.2: Backend Automated Testing Results

Route Tests

The `backend` test suite is organised under a `tests/routes/` directory, with one test file per route module. Table 5.1 presents the results for each route.

Testing `tests/routes/`	
Test File	**Result**
`friends.test.ts`	PASS
`gameInvites.test.ts`	PASS
`games.test.ts`	PASS
`requests.test.ts`	PASS
`users.test.ts`	PASS

Table 5.1: Test results for `tests/routes/`

Discussion of Backend Results

All five route modules — users, friends, requests, games, and game invites — passed their full test suites. This confirms that the REST API endpoints respond correctly to both valid and invalid inputs, that authentication middleware behaves as expected, and that CORS configuration permits the `frontend` to communicate with the `backend` without errors. The passing of `games.test.ts` and `gameInvites.test.ts` in particular provides evidence that the core game lifecycle — creation, invitation, acceptance, and completion — is handled reliably at the API level. As with the `frontend`, however, the `backend` tests focus on unit and route-level behaviour. Broader integration tests that exercise the interaction between multiple routes and services in sequence, as well as stress tests under concurrent load, would strengthen confidence in the system's behaviour under real-world conditions.

Deployment Testing

Beyond automated tests, the system was validated in a deployed, production-like environment by hosting the `frontend` and `backend` on separate cloud platforms.

Frontend Deployment — Vercel

The React `frontend` was deployed to **Vercel**, a platform optimised for modern JavaScript frameworks. Vercel's continuous deployment pipeline was connected to the GitHub repository, allowing every push to the main branch to trigger an automatic rebuild and redeployment. The deployment was verified by navigating to the live URL and confirming that all seven pages loaded correctly, that routing between pages functioned as expected, and that the application prompted users to install it as a Progressive Web App (PWA) on supported devices. Environment variables such as the `backend` API URL and any third-party service keys were configured securely through Vercel's project settings, keeping sensitive values out of the repository.

Backend Deployment — Railway

The Node.js/Express.js `backend` was deployed to **Railway**, a cloud platform well-suited to persistent server workloads. Railway was configured to pull from the `backend` directory of the GitHub repository and to inject the required environment variables — including database connection strings and authentication secrets — at runtime. The deployment was verified by issuing HTTP requests to each of the API routes from the deployed `frontend`, confirming that user registration, login, friend management, and game creation all operated correctly in the cloud environment.

The database, also provisioned through Railway, maintained persistent state across requests as expected.

Evaluation Against Functional Objectives

This section assesses the degree to which the implemented system meets the functional objectives set out in the Introduction. The objectives are evaluated on the basis of the testing results presented above and **direct observation** of the deployed application.

Chess Gameplay Against a Grandmaster Engine

A central objective of the project was to allow users to play chess against an engine that **emulates** grandmaster-level thinking, with the aim of challenging and training players to think more deeply about their moves. This was achieved through integration with **Stockfish**, one of the world's strongest open-source chess engines. The `ChessBoard`, `GameController`, and `GameScreen` components — all of which passed their automated tests — collectively implement the full game loop: move validation, engine response, position evaluation, and game termination. The timer component ensures that timed game formats are supported, adding competitive pressure consistent with real tournament conditions.

Move Accuracy Tracking and Feedback

A key training objective was to provide players with feedback on the quality of their moves, rather than simply recording wins and losses. This was addressed through the `AccuracyStats` component, which passed its automated test and is responsible for computing and displaying move accuracy scores. By comparing the player's chosen move against the engine's preferred continuation at each step, the system quantifies how closely the player is thinking like a grandmaster. This feature directly supports the educational purpose of the platform.

Social and Multiplayer Features

The project aimed to create a platform rather than a standalone tool, which required social features enabling users to connect, challenge, and play against one another. The Friends subsystem — comprising `AddFriends`, `FriendsList`, `PendingRequests`, `SentRequests`, `ChallengeDialog`, `GameInvites`, `GameDialog`, `ChatDialog`, and `ProfileDialog` — represents a substantial portion of the application and passed all of its tests. The corresponding `backend` routes (`friends.test.ts`,

`requests.test.ts`, `gameInvites.test.ts`) also passed in full, confirming that the social graph and invitation system operate correctly end-to-end.

Tutorial and Learning Content

To cater for players who are new to chess or who wish to consolidate their understanding of the rules and piece movements, a dedicated `Tutorial` section was implemented. **All eight** tutorial component tests passed, covering basic rules, individual piece guides, drawing conditions, and winning conditions. This provides evidence that the learning content is rendered correctly and that the interactive mini-board used for illustration is functional.

User Profiles and Progress Tracking

The system maintains persistent user profiles, storing game history and statistics that allow players to track their improvement over time. The `Profile` page and associated `backend users` route both passed their respective tests, and the deployment to Railway confirmed that data persists correctly across sessions in the cloud-hosted database.

Progressive Web App (PWA) Support

An additional objective was to make the application installable on mobile devices via PWA support, broadening accessibility beyond desktop browsers. The `InstallPWA` component passed its test, and the deployment to Vercel confirmed that the PWA manifest and service worker were served correctly, enabling installation on both Android and iOS devices.

5.2 Limitations and Opportunities for Future Work

Identifying limitations is not a sign of weakness — it is evidence of critical insight into the decisions made throughout the project. The following reflects honestly on areas where the system falls short of an ideal implementation and where future iterations could improve upon the current work.

Absence of End-to-End Tests

The most significant gap in the current test strategy is the lack of end-to-end (E2E) tests. While the 132 `frontend` and 116 `backend` tests provide strong coverage at the unit and route level, they do not exercise complete user journeys — for example,

a user registering, navigating to the Play page, completing a game against the engine, and reviewing their accuracy score. A tool such as **Cypress** or **Playwright** would enable this level of testing and would catch integration failures that unit tests cannot detect. This is a concrete and high-value area for future improvement.

Stockfish Integration Constraints

The current integration with Stockfish runs the engine at a fixed or lightly configurable difficulty level. A more sophisticated implementation would expose graduated difficulty settings that more meaningfully adapt to the player's current skill level, providing a progressive challenge curve. Additionally, the depth and time budget allocated to Stockfish's search may create perceptible latency on slower devices or network conditions; optimising this — for example by running the engine in a Web Worker on the client side — would improve the responsiveness of the gameplay experience.

Limited Real-Time Infrastructure

Real-time features such as live game challenges, in-game chat, and opponent move updates are implemented using periodic polling or basic event handling. A more scalable and responsive architecture would use **WebSockets** (e.g. via **Socket.IO**) to push updates to clients instantly rather than relying on polling intervals. This is particularly relevant for the multiplayer game invitation flow, where delays in receiving a challenge notification could degrade the user experience.

Database Scalability

The current database schema and query patterns were designed for a small number of concurrent users typical of a final-year project. Under significant concurrent load — many simultaneous games, friend requests, and chat messages — query performance may degrade without indexing strategies and connection pooling tuned for scale. Future work could include load testing with a tool such as **k6** or **Artillery** to identify bottlenecks and implement appropriate optimisations.

Accessibility

Chess interfaces present non-trivial accessibility challenges: a visual board with interactive pieces is inherently difficult to make screen-reader-friendly. The current implementation does not include a dedicated accessibility mode (such as a text-based board representation or keyboard-navigable move input). Addressing this in

a future iteration would broaden the potential user base and align the application with **WCAG 2.1** guidelines.

Grandmaster Move Database

The application's name and concept — *Guardians of the Chess Grandmaster* — implies a comparison between the player's moves and those made by real grandmasters in historical games. An opportunity for future work would be to integrate an opening book or a database of annotated grandmaster games (such as those available from **Lichess** or **Chess.com** APIs), allowing the system to contextualise the player's decisions within the broader canon of expert play rather than relying solely on engine evaluation.

Summary

This chapter has demonstrated that the *Guardians of the Chess Grandmaster* system is robust and functionally complete with respect to its core objectives. All 248 automated tests (132 `frontend`, 116 `backend`) passed, and the application was successfully deployed to production-grade cloud infrastructure on Vercel and Railway. The system delivers chess gameplay against a Stockfish engine, move accuracy feedback, social and multiplayer features, a structured tutorial, persistent user profiles, and PWA support — fulfilling each of the functional objectives set out at the outset of the project. The limitations identified — notably the absence of end-to-end tests, real-time WebSocket infrastructure, and an accessibility mode — provide a clear roadmap for a next iteration of the platform.

Chapter 6

Conclusion of GOTCG

6.1 Context and Objectives

Chess is one of the oldest and most **extensively** studied **strategy** games in the world, yet there remains a significant gap between a **casual player** and a **grandmaster-level** thinker. Although resources like puzzle books, video tutorials, and engine analysis are available, few platforms allow learners to step into the shoes of elite players and challenge themselves to think at that level. This lack of **effective pedagogical tools** in chess inspired the development of Guardians of the Chess Grandmaster (GOTCG).

The primary objective of this project was to design and develop an interactive web application that presents players with carefully **curated positions** drawn from actual grandmaster games. Players are required to identify the **optimal move** before the master's choice is revealed. This approach is grounded in the educational principle of deliberate practice: by repeatedly engaging with **high-quality** positions and receiving immediate, contextual feedback, users can develop deeper positional intuition and tactical awareness over time.

Several concrete engineering objectives underpinned this goal. The platform needed to provide a responsive and **intuitive** chess interface accessible via modern web browsers, a curated and extensible database of grandmaster positions, a reliable **move-validation** and evaluation engine, a meaningful scoring and progression system, and a user authentication layer to track progress across sessions. Together, these components formed the foundation of the GOTCG experience.

6.2 Population of People Who Played GOTCG

To contextualise the potential reach of GOTCG, it is worth considering the scale of the existing chess-playing population from which its user base would be drawn.

Figure 6.1: Chess.com World Map Population

Chess.com, the world's largest online chess platform, reports a global membership spanning every continent, with particularly **strong representation** in the United States (43.9 million members), India (20.4 million), the United Kingdom (8.9 million), the Philippines (7.3 million), and Brazil (7.1 million), as illustrated in Figure 6.1 below.

Conservatively estimating that **approximately 10%** of active Chess.com players represent the **target demographic** for a grandmaster-training tool such as GOTCG — players motivated enough to seek structured improvement beyond casual games — yields a **prospective** audience in the tens of millions worldwide. Even a modest penetration of this segment would represent a substantial and geographically **diverse** user base, underscoring both the commercial viability and the broad educational impact that GOTCG could realistically achieve.

6.3 Summary of Findings from System Evaluation

The System Evaluation chapter involved a combination of functional testing, performance profiling, and user acceptance testing of GOTCG. The results of this evaluation are summarized below.

- **Core Gameplay Loop Validated: All test scenarios** (for example Ruby scenarios from Software Engineering) confirmed that the platform accurately loads grandmaster positions, accepts and validates user moves through the

integrated chess engine, and displays the grandmaster's move alongside an explanation. No critical defects were identified in the move-handling process.

- **Responsive UI Performance:** The chessboard interface rendered correctly across Chrome, Firefox, and Safari on both desktop and mobile devices. The average page load time remained below two seconds on a standard **broadband** connection, meeting the non-functional requirements set at the outset.

- **Scoring System Accuracy:** The **point-awarding** mechanism accurately distinguished between exact grandmaster matches, engine-approved alternatives, and suboptimal responses. It produced a scoring scheme that rewarded precision while not unduly penalizing creative yet sound play.

- **User Authentication and Persistence:** Account creation, login, and session management functioned correctly in **all tested** scenarios. User scores and completed puzzle sets persisted reliably across sessions and devices.

- **User Acceptance Testing:** Volunteer testers of **varying skill levels** participated in the evaluation, providing valuable feedback.

6.4 Opportunities for Future Investigation

The development of GOTCG has revealed few concrete directions for future work:

- **Spaced-Repetition System:** Integrating a spaced-repetition algorithm, similar to those used in language-learning applications, would allow the platform to **resurface positions** that individual users consistently find challenging. This would reinforce learning through optimal review intervals.

- **Expanded and Community-Contributed Position Database:** By opening the curation pipeline to verified contributors, we could dramatically increase the breadth and diversity of available grandmaster positions. This expansion would cover a wider range of openings, **endgame** scenarios, and historical eras.

- **Adaptive Difficulty:** Leveraging the engine-latency findings discussed earlier, a **future iteration** could automatically adjust the difficulty of the positions presented to each user based on their demonstrated skill level. This would provide a more personalized learning experience. For example on Figure 6.2

Figure 6.2: Six AI difficulty levels, including a special level

6.5 Closing Remarks

Guardians of the Chess Grandmaster aims to fill a significant gap in chess education by **immersing learners** in the decision-making processes of the world's top players. The system developed through this project achieves this by presenting **authentic** grandmaster positions, challenging users to think at a high level, and immediately contextualising the results with the reasoning of the grandmasters themselves.

This project has been both technically and intellectually rewarding. Building a full-stack web application that integrates a chess engine, manages user states, and delivers a **polished interactive** experience required knowledge from various areas of the computing curriculum, including database design, RESTful API development, `frontend` engineering, and **software testing** methodologies. The challenges encountered along the way, along with the solutions devised to address them, have led to the creation of a platform that is both functional and genuinely enjoyable to use.

Chess teaches patience, foresight, and the discipline to consider consequences several moves in advance. Similarly, building Guardians of the Chess Grandmaster demanded these **same qualities**. The result is a platform with a **strong foundation** and a well-defined path for growth. With continued development, it has the potential to become a meaningful tool for **aspiring** players who wish to think like grandmasters.

Screencast Demonstration

YouTube Link: GOTCG Screencast Link

GitHub Repository Link

tomaspettit2506/final-year-project

Bibliography

[1] M. Gackenheimer. *Introduction to React*. Apress, Berkeley, CA, 2015.

[2] S. Aggarwal. Modern web-development using reactjs. *International Journal of Recent Research Aspects*, 5(1):133–137, 2018.

[3] P. Hunt and Facebook Engineering. React: A javascript library for building user interfaces. In *Proceedings of JS Conf*, 2016.

[4] G. Bierman, M. Abadi, and M. Torgersen. Understanding typescript. In *ECOOP 2014 – Object-Oriented Programming*, pages 257–281, Berlin, 2014. Springer.

[5] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in javascript. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 758–769, 2017.

[6] E. You. Vite vs webpack: A performance comparison, 2023. ViteJS Blog.

[7] Stefan Tilkov and Steve Vinoski. Node.js: Using javascript to build high-performance network programs. *IEEE Internet Computing*, 14(6):80–83, 2010.

[8] I. Chaniotis, K. Kyriakou, and N. Tselikas. Is node.js a viable option for building modern web applications? a performance evaluation study. *Computing*, 97(10):1023–1044, 2015.

[9] C. Pautasso, O. Zimmermann, and F. Leymann. Restful web services vs. big web services: Making the right architectural decision. In *Proceedings of the 17th International Conference on World Wide Web*, pages 805–814, 2008.

[10] N. Nurseitov, M. Paulson, R. Reynolds, and C. Izurieta. Comparison of json and xml data interchange formats: A case study. In *Proceedings of the 2009 ISCA 22nd International Conference on Computer Applications in Industry and Engineering*, 2009.

[11] Victoria Pimentel and Bradford G. Nickerson. Communicating and displaying real-time data with websocket. *IEEE Internet Computing*, 16(4):45–53, 2012.

[12] E. Brown. *Web Development with Node and Express*. O'Reilly Media, Sebastopol, CA, 2014.

[13] A. Mardan. *Pro Express.js*. Apress, Berkeley, CA, 2014.

[14] K. Chodorow. *MongoDB: The Definitive Guide*. O'Reilly Media, Sebastopol, CA, 2 edition, 2013.

[15] Z. Parker, S. Poe, and S. V. Vrbsky. Comparing nosql mongodb to an sql db. In *Proceedings of the 51st ACM Southeast Conference*, pages 1–6, 2013.

[16] C. Strauch, U.-L. S. Sites, and W. Kriha. Nosql databases, 2011. Lecture Notes, Stuttgart Media University.

[17] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[18] A. Rodriguez. Restful web services: The basics, 2008. IBM developerWorks.

[19] R. Fielding and J. Reschke. Hypertext transfer protocol (http/1.1): Semantics and content. RFC 7231, 2014.

[20] D. Crockford. The application/json media type for javascript object notation (json). RFC 4627, 2006.

[21] ECMA International. The json data interchange syntax, 2017.

[22] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). RFC 7519, 2015.